LAB-5 – Interface Patterns in Practice

Extension of LAB-4: Dependency Isolation

Goal

Extend the architecture created in LAB-4 by implementing real interface patterns used in software systems. Students will apply Strategy, Repository, Service Abstraction, and Adapter patterns using interfaces.

General Requirements

• Program.cs works only with interfaces

• No business logic in Main method

• Dependencies are passed via constructor injection

• At least two implementations per interface

• Demonstrate runtime behavior switching

• README must explain which pattern is implemented and why

Variant 1 – Student Printing (Strategy Pattern)

Interface: IStudentPrinter

Create implementations:

• ConsoleStudentPrinter

• FileStudentPrinter

• JsonStudentPrinter

Demonstrate switching printing behavior without modifying StudentService.

Variant 2 – Average Calculation (Strategy Pattern)

Interface: IAverageStrategy

Create implementations:

• SimpleAverageStrategy

• WeightedAverageStrategy

• MedianAverageStrategy

Demonstrate running the program with different calculation strategies.

Variant 3 – Menu Service (Service Abstraction)

Interface: IMenuService

Create implementations:

• ConsoleMenuService

• DebugMenuService

• WebMenuSimulationService

Explain how service abstraction isolates user interaction from business logic.

Variant 4 – Student Finder (Repository Pattern)

Interface: IStudentRepository

Create implementations:

• MemoryStudentRepository

• FileStudentRepository

• ApiStudentRepository (simulated)

Explain how Repository pattern abstracts data access.

Variant 5 – Validation (Adapter Pattern)

Interface: IStudentValidator

A legacy validation system exists. Create an adapter:

StudentValidatorAdapter

The adapter should translate the legacy validation interface into IStudentValidator.

Challenge – Architecture Integration

Create a final architecture combining multiple patterns.

Program → IMenuService → StudentService

StudentService → IStudentRepository

StudentService → IStudentPrinter

StudentService → IStudentValidator

StudentService → IAverageStrategy

Demonstrate runtime switching of implementations without modifying StudentService.

Evaluation (10 points)

3 – Correct use of interface pattern

2 – Multiple implementations

2 – Runtime behavior switching

1 – Clean constructor injection

2 – README explanation of architecture